# Computación gráfica y ambientes virtuales (0494)

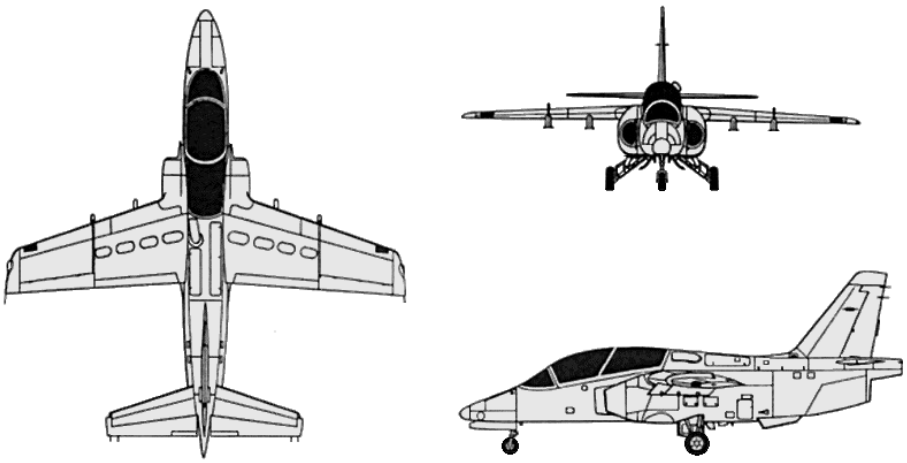
# Práctico 04

### Objetivo

El objetivo es **completar el armado de la aeronave** planteado en el taller de la clase: componer un único modelo usando varias primitivas del Práctico 03 y verificar esa composición sometiéndola a una **rotación conjunta**.

### Punto de partida

Los módulos `Mesh`, `Shader` y `primitives` (`cube()`, `cylinder()` y `cone()`) del Práctico 03 y `glm` para armar la matriz de modelo. Lo otro que hace falta para iniciar el práctico es el **despiece** hecho en la actividad de aula: qué primitiva aproxima cada parte del avión, con qué medidas y con qué transformaciones. A continuación se encuentran las tres vistas de la aeronave usadas en la actividad aúlica:



Tres vistas SIAI Marchetti/Aermacchi S-211.

El despiece que hayan propuesto es el punto de partida de este trabajo. No hay una solución única porque la cantidad de piezas y qué primitivas se usa en cada una dependen de este despiece (y es una propuesta personal).

## Parte 1 — El módulo `Aircraft`

Conviene que el modelo de la aeronave no quede suelto en el `main`: agruparlo desde el principio en un módulo propio, por ejemplo `Aircraft`, que sea el dueño de las piezas y de las matrices que las componen. El resto del programa no necesita saber cuántas primitivas tiene el avión ni cómo están armadas: sólo necesita, por cuadro, una lista de piezas ya transformadas y listas para dibujar. Esa separación es la que va a permitir, más adelante, reemplazar el modelo generado con primitivas por uno cargado desde un archivo (`.obj`) sin tocar el resto del programa.

```
// RenderItem.h --- punto de partida, no molde
struct RenderItem {
    const Mesh* mesh  = nullptr;         // malla a dibujar (referencia, no es propia)
    glm::mat4   model = glm::mat4(1.0f); // transformacion modelo -> mundo, YA compuesta
    glm::vec4   color = glm::vec4(1.0f);
};

// Aircraft.h --- punto de partida, no molde
class Aircraft {
public:
    void init();                                          // arma piezas y locales, UNA VEZ
    void update(glm::vec3 pos, glm::vec3 angulos);        // recalcula la pose, cada cuadro
    void collect(std::vector<RenderItem>& items) const; // agrega un item por pieza al vector

private:
    std::vector<Mesh>      piezas_;
    std::vector<glm::mat4> locales_;   // una por pieza, fijas desde init()
    glm::mat4              pose_ {1.0f};
};
```

**Funcionalidades que debe tener.**

- `init()` genera las piezas y calcula sus matrices locales **una sola vez**; no se vuelve a llamar a los generadores después (Parte 2).
- `update()` recalcula únicamente `pose_`, a partir de la posición y la orientación actuales de la aeronave; no toca las piezas ni las matrices locales (Parte 3).
- `collect()` agrega a `items` un `RenderItem` por pieza, con su transformación ya compuesta — pose y matriz local— y lista para dibujar (Parte 3). Es la **única** información que el resto del programa necesita para dibujar el avión.

**Algunas cuestiones a pensar.**

- Si se llama a `collect()` antes de haber llamado nunca a `update()`, ¿con qué pose se dibuja el avión? ¿Y si se llama antes de `init()`?
- ¿Por qué separar `init()` (una vez) de `update()` (cada cuadro)? ¿Qué costo tendría recalcular las piezas en cada cuadro?
- El día que las piezas se reemplacen por una malla cargada de un `.obj`, ¿qué método de `Aircraft` cambia? ¿Y qué parte del resto del programa tiene que cambiar?

## Parte 2 — Piezas y matrices locales (`init()`)

La creación del modelo de la aeronave de esta parte de la actividad es lo que debería estar / llamarse desde `Aircraft::init()`. Cada parte del avión se aproxima con una primitiva del Práctico 03 y una matriz **local**, construida desde la identidad, en coordenadas del modelo. Hay dos opciones para componer el modelo: **a)** usar una estructura jerárquica en donde cada pieza depende de otra pieza (la posición y orientación se da respecto a una pieza "padre") o **b)** cada pieza tiene su matriz de transformación que la ubica respecto al sistema de referencia del modelo (todo está referido a este sistema y no hay una estructura jerárquica de dependencia entre piezas).
Independientemente de la composición elegida todas las matrices locales se calculan al crear el modelo y no vuelven a cambiar mientras el avión no cambie de forma (no se están considerando partes móviles en el modelo que se plantea).

**Contenido mínimo del modelo.**

- El fuselaje planteado con dos o más primitivas
- El ala
- El grupo de cola: empenaje horizontal y empenaje vertical
- Ajustar la forma de cada pieza para tratar de aproximar lo mejor que se pueda la aeronave de referencia. Por ejemplo: se puede usar un **escalado no uniforme** (`glm::scale`) con un factor distinto por eje para transformar un cubo en una placa que represente un ala o un estabilizador.

**Algunas cuestiones a pensar.**

- Para un componente de la aeronave **simétrico respecto de su propio plano medio** — por ejemplo un ala recta— no se conviene aplicarle la transformación `glm::scale(m, glm::vec3(-1.0f, 1.0f, 1.0f))` (porque se altera el ordenamiento de los vértices —winding—), ese mejor aplicar una **traslación** de la misma pieza al lado opuesto.
- La normal que devuelve cada generador, ¿sigue siendo perpendicular a la superficie después de un escalado no uniforme? ¿Hay alguna pieza para la que la respuesta no importe?
- Contando piezas repetidas (por ejemplo el ala representada por dos placas): ¿cuántas **mallas** distintas hace falta generar, y cuántas **matrices locales** hay? ¿Por qué esos dos números no necesariamente tienen por qué coincidir?

## Parte 3 — *Pose* y verificación (`update()`, `collect()`)

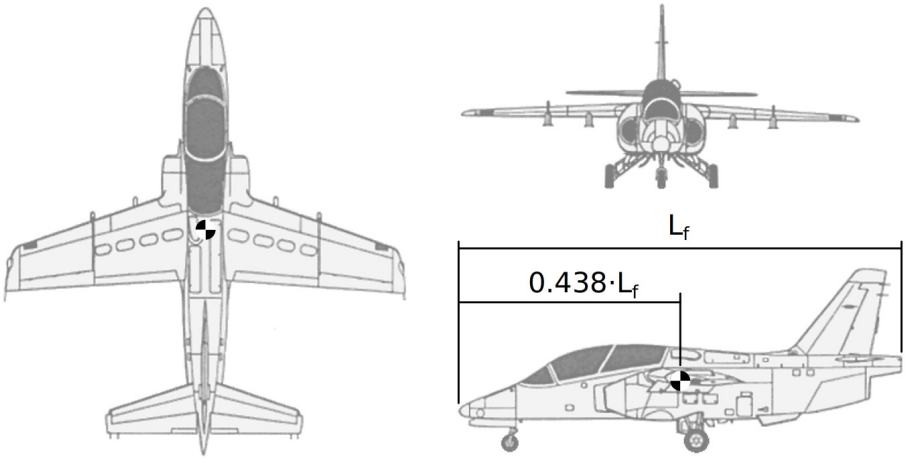
Esta parte de la actividad práctica es lo que debería ir en `Aircraft::update()` y `Aircraft::collect()`. Todas las piezas se ubican en el mundo con una misma matriz de *pose* que combina posición y orientación del avión:

$$M_{\text{mundo}}(\text{pieza}) \;=\; M_{\text{pose}} \cdot M_{\text{local}}(\text{pieza})$$

Los datos de la pose están referidos a un punto particular del modelo —el punto de referencia—que no es necesariamente el origen del sistema de referencia para el modelo. Antes de rotar hay que hacer coincidir ese punto con el origen:

$$M_{\text{pose}} \;=\; T(\text{posición}) \cdot R(\text{ángulos}) \cdot T(-\text{ref})$$

A continuación se muestra la posición del punto de referencia a utilizar para aplicar las transformaciones de *pose* al modelo de la aeronave.




Posición del punto de referencia para aplicar transformaciones de *pose*

El dibujado en sí sigue viviendo en el `main`: por cuadro, se llama a `update()` con la posición y los ángulos actuales, se junta la lista con `collect()` y se recorre para dibujar cada pieza.

```
// main.cpp --- dentro del bucle de dibujo, una vez por cuadro
avion.update(posicion, angulos);

std::vector<RenderItem> items;
avion.collect(items);

for (const auto& item : items)
    dibujar(item);   // set_uniform(model), set_uniform(color), glDrawElements(...)
```

**Funcionalidades que debe tener.**

- `update()` arma `pose_` con la composición de arriba, a partir de `pos` y `angulos`.
- `collect()` arma cada `RenderItem` con `model = pose_ * local`.
- **Verificación.** Desde el `main`, rotar la aeronave en uno de sus ejes (por ejemplo, un movimiento de cabeceo (subir / bajar la nariz): rotación respecto a un eje paralelo a la dirección del ala) y verificar que **todas** las piezas se mueven juntas, girando alrededor del punto de referencia elegido y no de un punto arbitrario del modelo.

**Algunas cuestiones a pensar.**

- ¿De qué lado de la composición va la traslación que lleva el punto de referencia al origen? ¿Qué le pasa al avión si se la pone del otro lado?
- Si una pieza tuviera que moverse por su cuenta —un alerón que se deflecta —, ¿alcanza con la misma matriz local fija? ¿Qué habría que cambiar?
- ¿Hace falta un árbol de transformaciones para este modelo? o ¿alcanza con una lista de piezas y dos niveles de jerarquía? ¿En qué caso dejaría de alcanzar?

**Cómo se sabe que está bien.**

- Al aplicar la *pose*, todas las piezas giran juntas alrededor del mismo punto: ninguna se despega y el conjunto no se corre de lugar.
- Ninguna pieza simétrica usa un escalado negativo.

---

Como en los prácticos anteriores, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno, y hay que poder explicarlas.